

AI ENGINEER INTERNSHIP · TECHNICAL CASE STUDY

Document Intelligence Platform

Intelligent extraction, financial validation and API delivery for invoices and financial statements

4

document types

138

automated tests

37

reconciliation checks

1

deployed service

The brief, and what was built

One deployed service that turns a document into structured, reconciled, retrievable data

WHAT WAS ASKED

- Accept PDF / JPG / PNG, up to 3 pages, and reject anything else gracefully
- Extract every visible field and table value, not a short fixed list with evidence
- Validate the financial relationships for each document type, per comparative period
- Store results and serve them through REST APIs and a dashboard
- Deploy it: a localhost-only submission is incomplete

WHAT WAS DELIVERED

- FastAPI service with four documented endpoints and Swagger UI
- Text layer or rasterise reader, then Claude vision with a schema constrained response
- Deterministic reconciliation engine - no model in the arithmetic
- PostgreSQL-backed dashboard with raw-JSON view
- Docker image and Render blueprint; no system packages required

Frontend and API are one process: one URL, one origin, no CORS.

The finding that shaped the design

Investigating the corpus before writing code changed the architecture

29 / 30

statement PDFs have no text layer and no embedded image

Their text is painted as vector path outlines.

Empty resource dictionary, no /Font, no /XObject — roughly a megabyte of path fill operators per page. pdfplumber and pypdf both return zero characters, on pages that look perfectly normal on screen.

A text layer only solution extracts nothing from 29 of the 30 files.

Rasterisation is the primary path

not a fallback bolted on for scanned documents

pypdfium2, not poppler

a self-contained wheel, so the container needs no system packages

Vision model over classical OCR

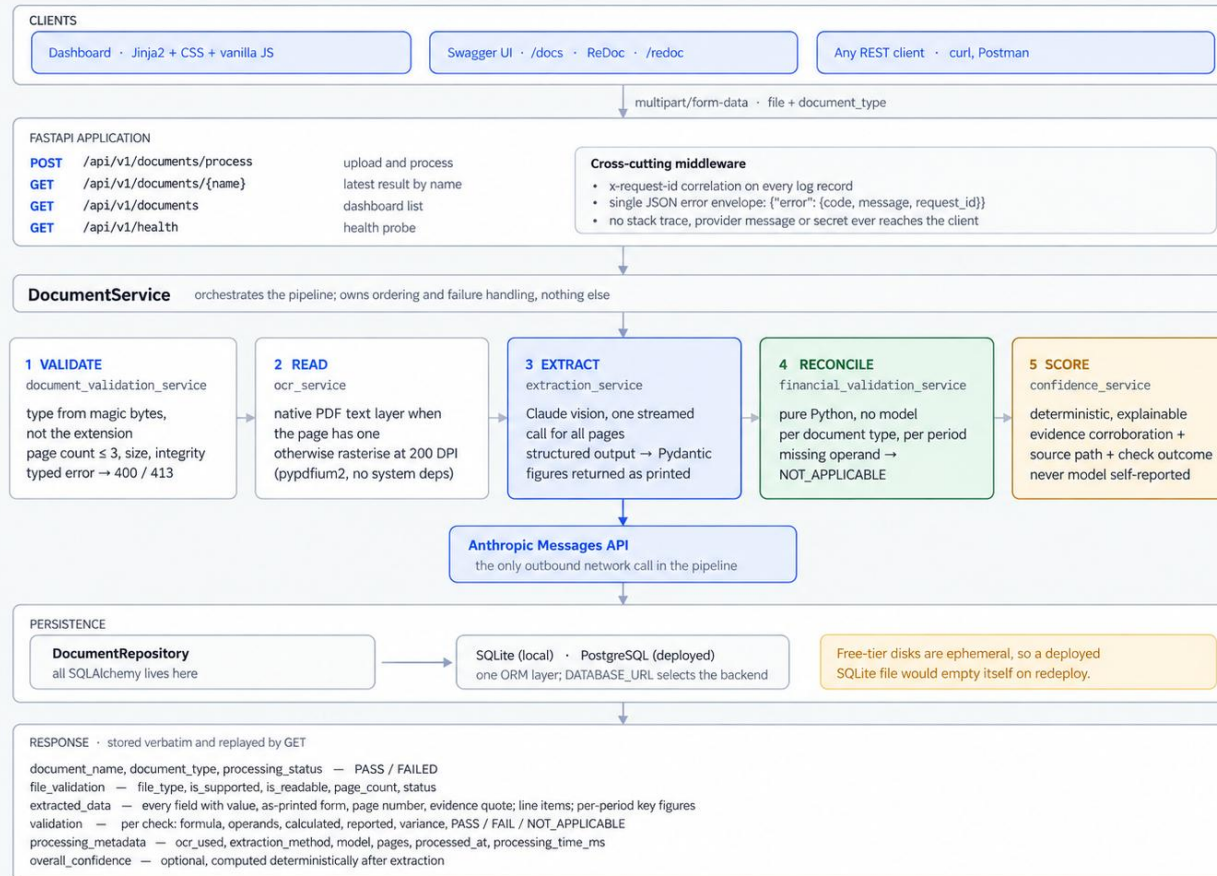
row-to-column association in dense comparative tables is exactly where Tesseract-class OCR fails

Architecture

Clear separation of concerns: each stage is one service with one job

Document Intelligence Platform

Extraction, validation and API service for invoices and financial statements



One process, two surfaces

The dashboard and the API are served by the same application, so there is one deploy, one URL and no CORS.

Each stage is swappable

The reader, the extractor and the reconciler share no state; each is constructor-injected and separately testable.

One external call

Only the extraction stage leaves the process. Everything else — validation, arithmetic, scoring — is local and deterministic.

The pipeline, in five stages

DocumentService owns the ordering and the failure handling; nothing else

1

Validate

document_validation_service

- Type from magic bytes, never the extension
- Empty, oversize, corrupted, > 3 pages
- Typed error → 400 / 413 / 422

2

Read

ocr_service

- Native text layer when the page has one
- Otherwise rasterise at 200 DPI
- EXIF rotation, downscale, JPEG encode

3

Extract

extraction_service

- One streamed Claude call, all pages
- JSON schema from Pydantic models
- Retry feeds the validator error back

4

Reconcile

financial_validation_service

- Pure Python — no model involved
- Per document type, per period
- Missing operand → NOT_APPLICABLE

5

Score

confidence_service

- Computed after the fact, not asked for
- Evidence corroboration is the key signal
- Absent value scores null, not zero

A rejected upload is still stored with processing_status FAILED, so the dashboard shows what was attempted rather than dropping it.

Extraction: the model transcribes, the code interprets

The single most important design decision in the system

THE MODEL RETURNS

- Every figure as a string, exactly as printed
- "(1,234.56)" "1,23,456.78" "-"
- The caption, section and page for each row
- The line of text each value came from
- null for anything it cannot read — never a guess

THE APPLICATION DECIDES

- Parentheses mean negative
- Indian and Western digit grouping
- Currency symbols, Unicode and trailing minus
- Dash / Nil / N-A mean no value, not zero
- Every reconciliation, to the last decimal

Why split it this way

Numeric interpretation stays deterministic and unit-tested instead of depending on the model doing arithmetic; the as-printed form is preserved for audit; and the confidence model can verify that a figure really appears in the evidence text the model quoted.

Financial validation

Deterministic, per comparative period, and never inventing an operand

Invoice / receipt

$\text{quantity} \times \text{unit_price} = \text{line amount}$

$\sum \text{line amounts} = \text{subtotal}$

$\text{subtotal} + \text{tax} - \text{discount} = \text{total}$

$\text{cash_paid} - \text{total} = \text{change}$

Balance sheet

$\text{total capital and liabilities} = \text{total assets}$

each section's components sum to that

section's own reported total

— derived from the document's structure

Profit and loss

$\text{interest earned} + \text{other income} = \text{total income}$

$\text{interest expended} + \text{opex} + \text{provisions}$

$= \text{total expenditure}$

$\text{income} - \text{expenditure} = \text{profit before MI}$

$\text{PBMI} - \text{minority interest} = \text{attributable}$

Cash flow

$\text{operating} + \text{investing} + \text{financing} + \text{FX}$

$= \text{net increase in cash}$

$\text{opening} + \text{net increase} + \text{cash acquired}$

$= \text{closing cash}$

Tolerance $|\text{calc} - \text{reported}| \leq 0.1 + 1\text{e-}5 \times \text{magnitude}$ — a combined allowance, both terms configurable

An OR of absolute-or-relative is too loose at both ends: a 0.5% relative limit passes a 20,000-crore discrepancy, while an absolute limit sized for a statement is 11% of a 9.00 till receipt. A missing operand returns NOT_APPLICABLE and never fails the document.

Confidence that means something

Optional in the brief — implemented, but never asked of the model

SIGNAL	WEIGHT
Source path A PDF text layer is exact; a rendered page was read visually.	base 0.75 / 0.60
Evidence corroboration The number is actually present in the quoted source text.	+0.20
Evidence completeness A page number and a verbatim quote were both returned.	+0.05 each
Validation agreement A figure inside a failing reconciliation is suspect.	+0.10 / -0.30

Why not ask the model?

A self-reported score is not evidence. It correlates with fluency, not with correctness, and it cannot be checked.

Every signal here is independently verifiable from the response itself — which is what makes the number explainable.

A field the document does not report scores null, not zero.

There is nothing to be confident about, and zero would read as “certainly wrong”. Values below 0.70 are highlighted in the dashboard.

API surface

Swagger generated from the same Pydantic models the endpoints return

POST	/api/v1/documents/process multipart upload; file + document_type
GET	/api/v1/documents/{document_name} latest stored result, case-insensitive
GET	/api/v1/documents dashboard list; filter, search, paginate
GET	/api/v1/health status, database, extraction_configured

One error envelope

```
{ "error": {  
  "code": "...",  
  "message": "...",  
  "request_id": "..." }}
```

14 typed codes. Stack traces and provider messages go to the log, never the body — asserted by a test.

THE STORED RESPONSE

- `extracted_data` — every field with its parsed value, as-printed form, page number and evidence quote; line items; per-period key figures; an explicit `missing_fields` list
- `validation` — per check: formula, operands, calculated, reported, variance, PASS / FAIL / NOT_APPLICABLE, and the tolerance applied
- `processing_metadata` — `ocr_used`, extraction method, model, pages, timestamp, elapsed milliseconds

The full response is persisted verbatim, so GET replays exactly what POST returned.

The dashboard

HTML, CSS and vanilla JS, calling the same public API



Dashboard API docs Health

01 — INTAKE

Process a document

PDF, JPG or PNG · up to 3 pages · 20 MB. Choose the document type, then run extraction and reconciliation.

PDF, JPG OR PNG UP TO 3 PAGES UP TO 20 MB

DOCUMENT TYPE

Invoice

DOCUMENT

Choose file

No file chosen

Process

02 — LEDGER

Processed documents

Showing 50 of 50 documents.

Search by name...

All types

Refresh

DOCUMENT	TYPE	STATUS	VALIDATION	CONFIDENCE	SOURCE	TIME	PROCESSED
Consolidated Balance Sheet 2026.pdf	Balance Sheet	PASS	PASS	85%	VISION / OCR	51.8s	11 Sept 2026, 22:34
Consolidated Profit & Loss 2026.pdf	Profit And Loss	FAILED	—	—	TEXT LAYER	13.4s	11 Sept 2026, 20:11

Upload, then every processed document with status, validation outcome and confidence.

Document Intelligence

Extraction · Validation · API

Dashboard API docs Health

— Back to dashboard

01 — SUMMARY

Consolidated Balance Sheet 2026.pdf

Balance Sheet · 1 page(s) · read via page rasterisation + vision model

PASS VALIDATION PASS 85%

FILE TYPE	PAGES	SOURCE	MODEL	CHECKS
application/pdf	1	Vision model	claude-opus-5	6 passed · 0 failed · 0 n/a
TOLERANCE	PROCESSING TIME	PROCESSED AT		
±0.1 + 0.00001 × magnitude	51.81 s	11 Sept 2026, 22:34		

03 — FIELDS

Extracted fields

Missing values are highlighted. Hover a value to see the source text it was read from.

FIELD	VALUE	AS PRINTED
Entity Name	HDFC Bank Limited HDFC Bank Limited Integrated Annual Report 2...	HDF
Statement Title	CONSOLIDATED BALANCE SHEET CONSOLIDATED BALANCE SHEET As at March 31, 2...	CONSOLIDATEE

03 — FIELDS

Extracted fields

Missing values are highlighted. Hover a value to see the source text it was read from.

FIELD	VALUE	AS PRINTED
Entity Name	HDFC Bank Limited HDFC Bank Limited Integrated Annual Report 2...	HDF
Statement Title	CONSOLIDATED BALANCE SHEET CONSOLIDATED BALANCE SHEET As at March 31, 2...	CONSOLIDATEE
Currency	₹ (₹ in crore)	
Units	in crore (₹ in crore)	
As At	As at March 31, 2026 As at March 31, 2026	As at
Note	The schedules referred to above form an integral part of the Consolidated Balance Sheet. The schedules referred to above form an inte...	The schedules referred to above form an integral part of the Consolidated
Auditor Reference	As per our report of even date. As per our report of even date.	As per our report
Auditor	For Batliboi & Purohit, Chartered Accountants, ICAI Firm Registration No.: 101048W For Batliboi & Purohit Chartered Accountants...	For Batliboi & Purohit, Chartered Accountants, ICAI Firm Registratio
Auditor	For B S R & Co. LLP, Chartered Accountants, ICAI Firm	For B S R & Co. LLP, Chartered Accountants, ICAI Firm Registration No.: 16

Per document: extracted fields, per-period tables, every check with its arithmetic, and the raw JSON.

Engineering discipline

The parts that do not show up in a screenshot

Testing

138 tests, zero network

The Anthropic client is stubbed throughout, so the suite runs the same in CI and with no API key.

Logging

Structured, correlated

Single-line JSON with an x-request-id on every record, returned in the response header and in error bodies.

Error handling

One envelope, 14 codes

Typed exceptions map to HTTP status; a test asserts no internal detail reaches the client.

Security

Bytes over claims

File type from magic bytes, filename sanitised against traversal, size ceiling before read, container runs unprivileged.

Configuration

Environment only

No secret has a default and none is committed; .env.example lists names, never values.

Modularity

One job per module

Validation, reading, extraction, reconciliation, scoring, persistence and routing are separate and separately testable.

Deployment

One Docker image, no system packages, one service serving both frontend and API

WHY THE IMAGE IS SIMPLE

- Every dependency ships as a binary wheel
- No apt-get layer, no poppler, no compiler
- python:3.12-slim, runs as an unprivileged user
- render.yaml provisions service + database

THE TRAP WORTH NAMING

Free-tier container disks are ephemeral.

A deployed SQLite file is wiped on every redeploy and on cold start, so the dashboard would quietly empty itself part-way through an evaluation. The ORM layer is backend-agnostic; only DATABASE_URL changes, and the blueprint points it at managed PostgreSQL.

VERIFIED AFTER DEPLOY

- /api/v1/health returns ok, database connected, extraction_configured true
- Swagger UI loads; POST processes a real document; GET by name returns it; the list backs the dashboard
- An unsupported file returns 400 with the documented error envelope and no stack trace

Limitations, and what production would need

Stated plainly — knowing where the edges are is part of the work

KNOWN LIMITATIONS

- Accuracy is bounded by the vision model. The reconciliations catch internally inconsistent extractions; they cannot catch a value that is wrong self-consistently.
- The caption registry is English and oriented to the statement conventions in scope; an unusual caption leaves a check NOT_APPLICABLE rather than guessing.
- Processing is synchronous — 15 to 40 seconds per rasterised page. Fine for this brief, not for real concurrency.
- Confidence is an explainable heuristic, not calibrated against labelled ground truth.
- No authentication; the evaluation flow needs an open API.

WHAT I WOULD CHANGE FOR PRODUCTION

- Asynchronous processing: 202 Accepted + a job id, a queue, and streamed progress.
- Content-hash caching so an identical re-upload costs nothing.
- Store the source document and link evidence to bounding boxes, so the UI highlights the exact region.
- An accuracy eval harness in CI — labelled corpus, field-level precision and recall — so a prompt change cannot silently regress extraction.
- Auth, per-tenant isolation, rate limiting and a model spend cap.
- Alembic migrations, OpenTelemetry traces, per-request cost accounting.

Thank you

Happy to walk through the code, the prompts, the validation logic or the deployment